

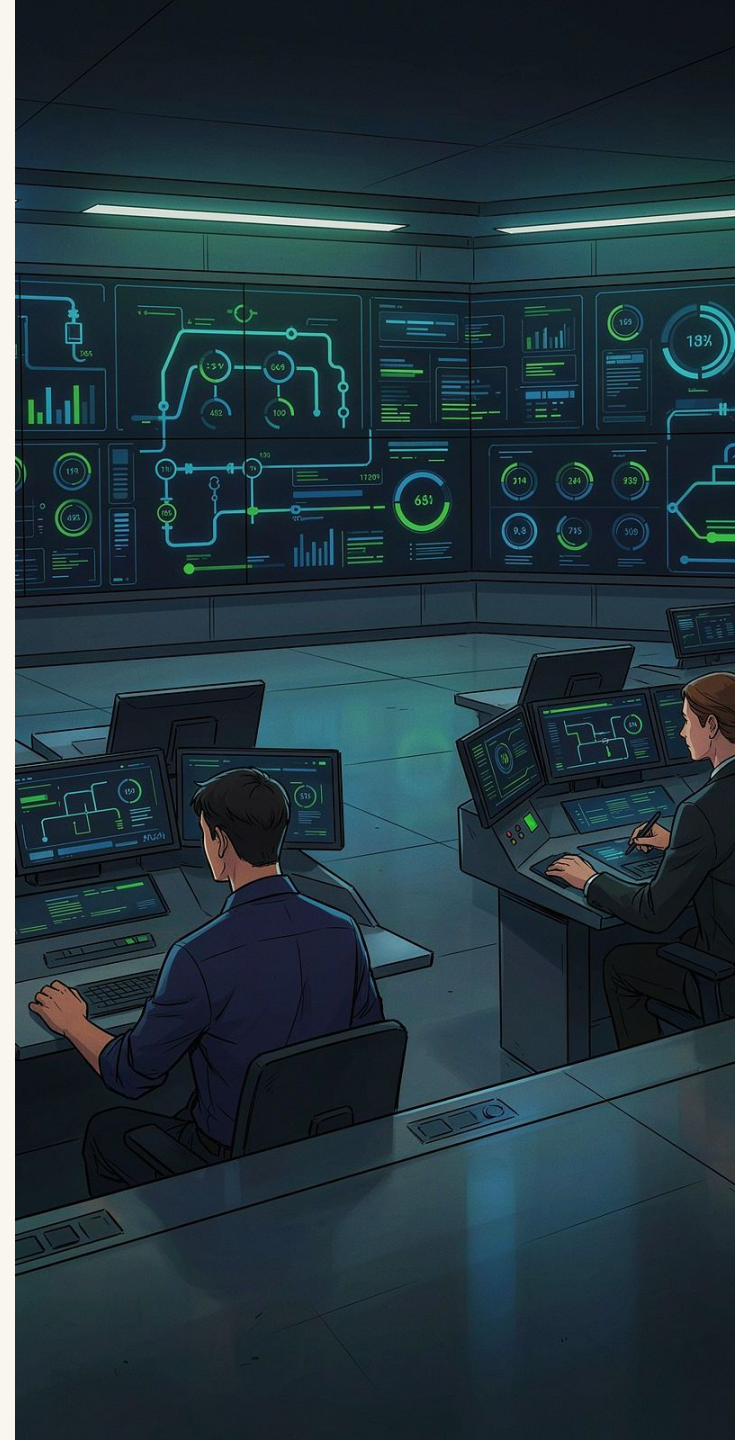
Module 8 – Harness CI

From Jenkins CI to Modern Cloud-Native CI

This module bridges what you already know — Jenkins, Kubernetes, and EKS Anywhere — with Harness CI, a modern software delivery platform. Every concept is mapped to your existing Jenkins knowledge so you can hit the ground running.

EKS ANYWHERE BATCH

MODULE 8



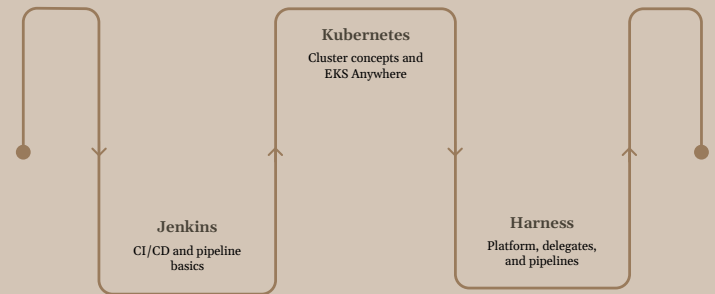
Learning Objectives & Training Approach

By the end of this module, you will understand:

- **Harness Architecture** — Platform, Delegates, Connectors, Secrets
- **Harness CI Pipelines** — Stages, Steps, Pipeline-as-Code
- **Templates** — Reusable, standardized pipeline configuration
- **Harness + Kubernetes** — Running Delegates inside EKS Anywhere
- **Harness + ArgoCD** — CI and GitOps CD as complementary tools
- **Jenkins → Harness** — Mapping every concept you already know

Training Approach

Your existing knowledge:



Every new Harness concept is introduced by mapping it to a Jenkins equivalent you already use daily.

Why Harness When We Already Know Jenkins?

Both Jenkins and Harness automate software delivery — but they operate at fundamentally different levels. Understanding this distinction is the first step.

Jenkins — Automation Server

- Developer commits code
- Git triggers Jenkins Controller
- Jenkins Agent picks up the job
- Build / Test / Package executes
- Artifact produced

Harness — Software Delivery Platform

- Developer commits code
- Git triggers Harness Platform
- Delegate executes inside your environment
- Build / Test / Security Scan / Deploy
- Artifact → Kubernetes / EKS Anywhere

📌 **Key Distinction:** Jenkins is primarily an automation server. Harness is a software delivery platform with integrated CI/CD, security scanning, governance, and deployment capabilities built in.

Jenkins vs. Harness: Mental Model

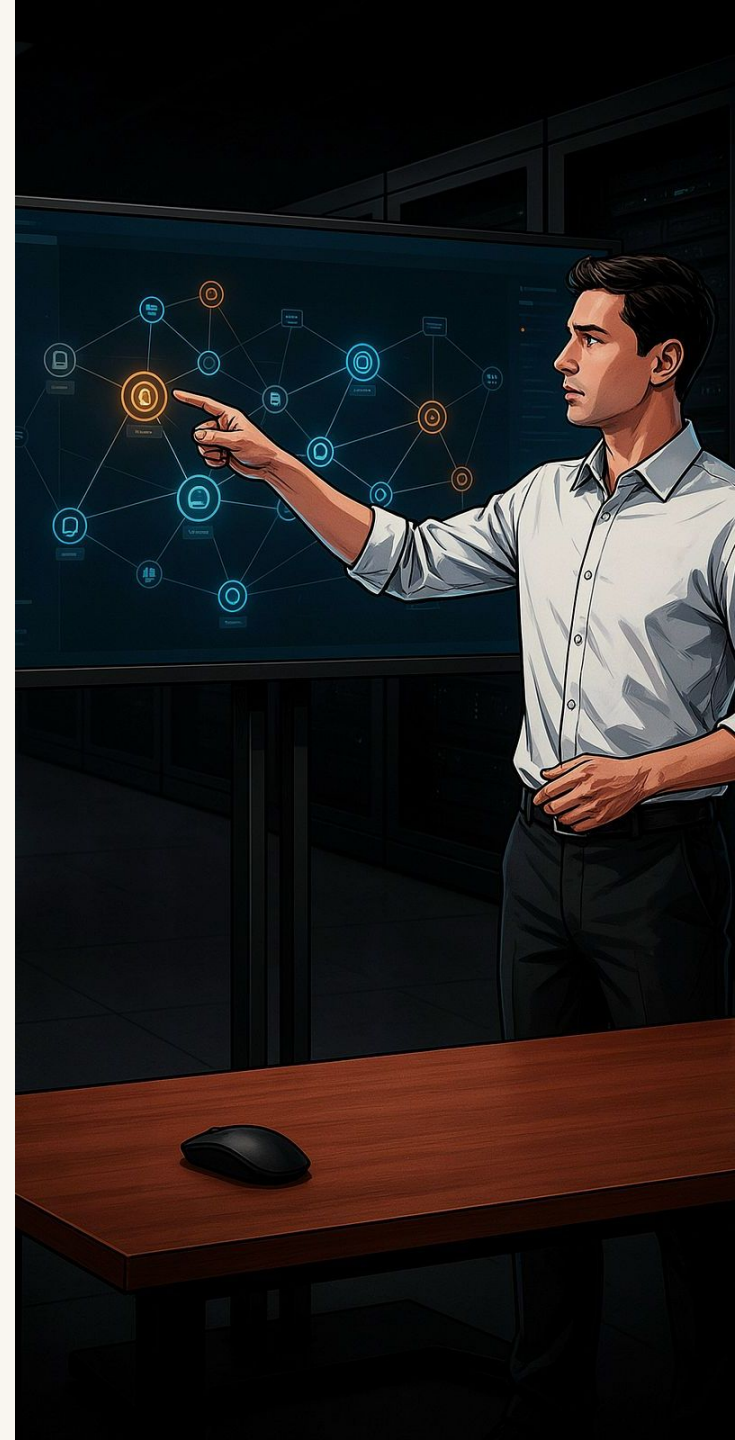
Before diving into architecture, lock in this mental model. Every Jenkins concept has a direct Harness counterpart — the terminology changes, but the underlying purpose is familiar.

Jenkins	Harness
Jenkins Controller	Harness Platform
Jenkins Agent	Harness Delegate
Jenkinsfile	Pipeline YAML
Credentials	Secrets / Connectors
Pipeline Stage	Stage
Pipeline Step	Step
Shared Library	Template
Agent Label	Delegate Selector
SCM Plugin / Config	Connector
Console Output	Execution Details

❏ **Do not think:** "Harness is just another Jenkins." Think: "**Harness provides a different operating model for software delivery.**"

Harness Architecture

Understanding the two-layer architecture is the foundation of everything else in this module. The platform orchestrates — the Delegate executes.



Harness Architecture: Deep Dive

Architecture Overview



Architecture Principle

Harness manages the orchestration from the cloud. The **Delegate performs operations inside your target environment** — which is critical for private EKS Anywhere clusters.

This separation means:

- Harness SaaS never needs direct access to your cluster
- The Delegate makes all internal API calls on behalf of Harness
- Your network firewall only needs to allow outbound HTTPS

Harness Platform

The Harness Platform is the cloud-hosted control plane — analogous to the Jenkins Controller, but with significantly broader capabilities built in rather than bolted on via plugins.

What Harness Platform Provides

Pipelines

Define, visualize, and manage all delivery workflows

Connectors

Integrations to GitHub, Docker, Kubernetes, AWS, Azure

Secrets

Centralized secret management with external store support

Templates

Reusable, versioned pipeline building blocks

RBAC & Governance

Role-based access, approval gates, audit trails

Execution History

Full pipeline run history, logs, and visualization

Jenkins Comparison

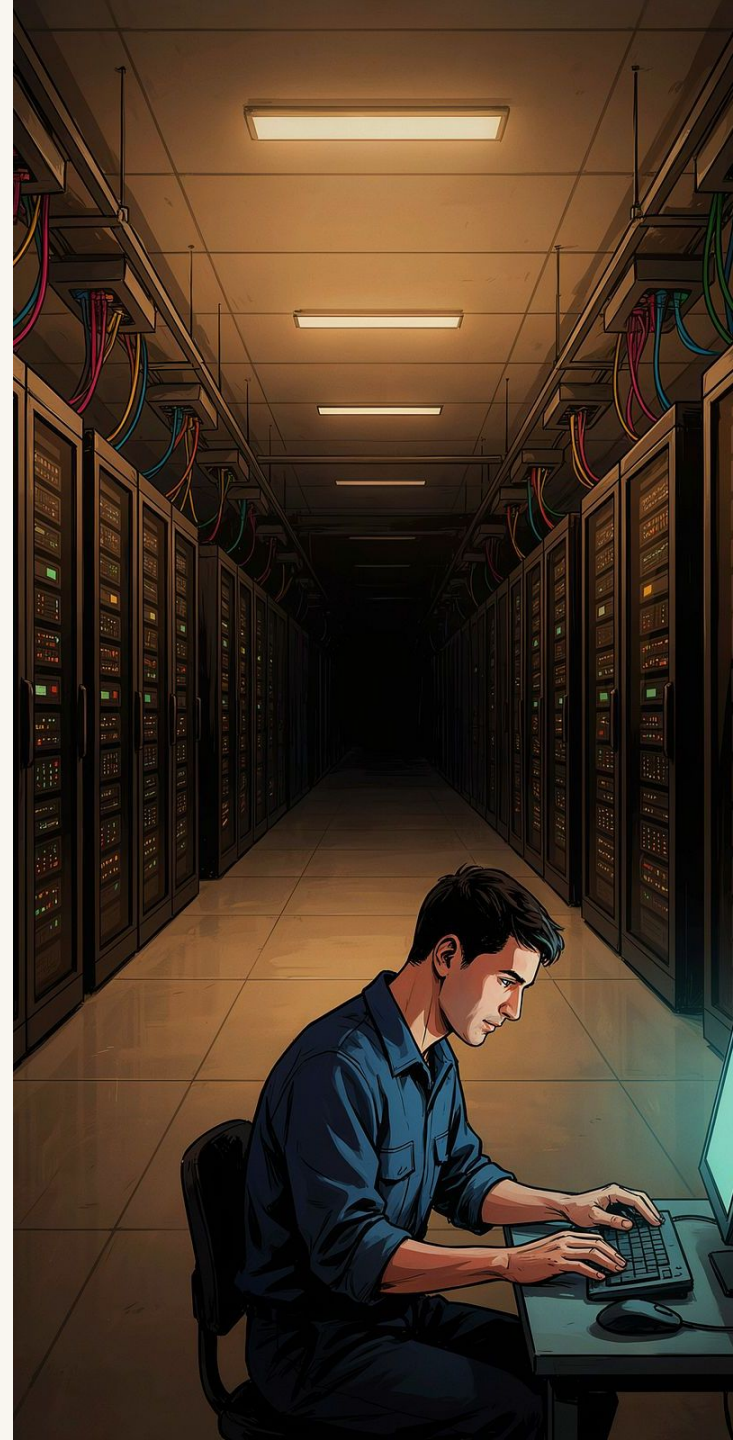
In Jenkins, you assemble these capabilities by installing and configuring plugins. In Harness, they are first-class platform features.

Jenkins Controller	Harness Platform
Jobs	Pipelines
Credentials	Secrets + Connectors
Plugins	Native Integrations
Agents	Delegates

The Harness Delegate

The Most Important Concept for EKS Anywhere Environments

A Harness Delegate is an agent that runs **inside your own environment**. It reaches out to Harness SaaS — Harness never reaches in. This is the architectural foundation for private EKS Anywhere clusters.



Harness Delegate: Architecture & Behavior

How the Delegate Works

01

Delegate is deployed into EKS Anywhere

Runs as a Kubernetes Deployment inside your private cluster

02

Delegate connects outbound to Harness SaaS

Uses HTTPS — no inbound ports required through your firewall

03

Harness assigns pipeline tasks to the Delegate

Based on Delegate Selectors that match pipeline configuration

04

Delegate executes tasks inside the private environment

Accesses GitHub, Container Registry, and Kubernetes API internally

Jenkins Equivalent

Harness Delegate $\approx$ Jenkins Agent

But the communication model is different:

- **Jenkins Agent:** Controller reaches inbound to the agent, or agent uses JNLP
- **Harness Delegate:** Always initiates outbound — Harness never reaches into your network

The Delegate can execute:

- `kubectl` commands
- `helm` operations
- `docker` builds
- Maven builds
- Git operations

Why Delegates Matter for EKS Anywhere

EKS Anywhere clusters are often on-premises, behind firewalls, and not reachable from the public internet. The Delegate solves this problem elegantly.

❌ Without Delegate (Problematic)

Harness SaaS → Internet

Harness would need direct access to your Kubernetes API

Firewall Barrier

Private EKS Anywhere is behind pfSense/firewall — not reachable from the internet

Connection Fails

Harness cannot reach the internal Kubernetes API endpoint

✅ With Delegate (Correct)

Harness SaaS → Task Assignment

Harness queues tasks — does not initiate connections inbound

Delegate → Outbound HTTPS

Delegate inside EKS Anywhere polls Harness for tasks — outbound only

Delegate → Internal API

Delegate calls the Kubernetes API from inside the private network

📌 **Security Principle:** The Delegate executes operations close to the infrastructure. This is the recommended pattern for all private and on-premises EKS Anywhere environments.

Delegate vs. Jenkins Agent: Side-by-Side

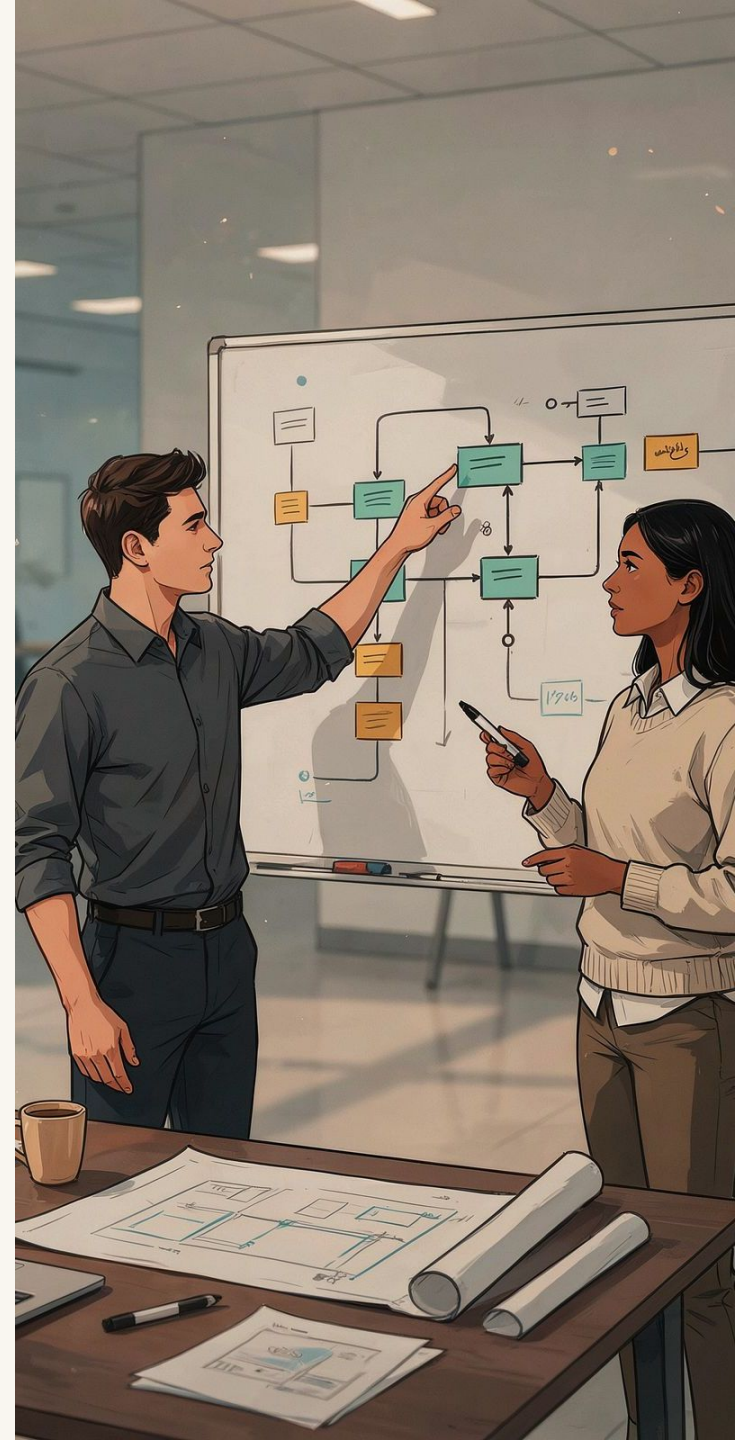
Capability	Jenkins Agent	Harness Delegate
Primary Role	Executes Jenkins jobs	Executes Harness pipeline operations
Connects To	Jenkins Controller	Harness Platform (SaaS)
Deployment Options	VM, Container, Bare Metal	Kubernetes Pod, VM, Docker
Targeting	Agent Labels	Delegate Selectors
Communication	Inbound or JNLP	Outbound HTTPS only
Managed Via	Jenkins UI	Harness Platform + Kubernetes

For EKS Anywhere, deploy the Delegate as a Kubernetes Deployment inside the harness-delegate namespace. It then has internal access to kubectl, helm, Maven, Docker, and the Kubernetes API — all from within your private network.

Harness Connectors

How Harness Connects to the Outside World

A **Connector** defines how Harness authenticates and communicates with an external system. Connectors replace the combination of Jenkins plugins, credentials, and inline pipeline configuration.



Connectors: Concept & Examples

Common Harness Connectors



GitHub Connector

Authenticates to GitHub repositories using a token or SSH key. Used by pipelines to clone source code.



Container Registry Connector

Connects to Docker Hub, ECR, or a private registry to push and pull container images.



Kubernetes Connector

Connects to the EKS Anywhere Kubernetes API for deploying workloads and running pipeline steps.

Jenkins Comparison

In Jenkins, you achieve similar results by combining:

- Credentials (username/password, tokens)
- SCM Plugin configuration
- Inline pipeline code (`credentialsId`)
- Per-plugin configuration

In Harness, these are unified as explicit, named **Connectors** — reusable across all pipelines and auditable in the platform UI.

Connector Example: GitHub

Let's walk through a real example. Your `student-management` Spring Boot application lives in GitHub. Harness needs a Connector to access it.

Jenkins Approach

Credentials are embedded directly in the Jenkinsfile:

```
pipeline {
  agent any
  stages {
    stage('Clone') {
      steps {
        git credentialsId: 'github-creds',
           url: 'https://github.com/
              org/student-management'
      }
    }
  }
}
```

Credential ID is referenced inline — pipeline and connection config are coupled.

Harness Approach

Create a named **GitHub Connector** once in the platform:

- Define **GitHub Connector** with token/SSH auth
- Pipeline references the Connector by name
- Delegate uses the Connector to clone the repository
- Connection config is **decoupled** from pipeline code

Updating credentials means changing the Connector once — all pipelines pick up the change automatically.

Secrets Management

Secrets should **never** live inside pipeline code or be committed to Git. Harness enforces a clean separation between pipeline logic and sensitive credentials.

What Counts as a Secret?

GitHub Token Personal access token for repository access	Registry Password Docker Hub or private registry credentials
Database Password Application database connection credentials	API Token External service authentication keys
SSH Key Key-based authentication for servers and Git	TLS Certificate SSL/TLS certificates for secure communication

Jenkins vs. Harness Secret Flow

Jenkins	Harness
Manage Jenkins → Credentials	Harness Platform → Secrets
Credential ID in Jenkinsfile	Secret Identifier in Pipeline YAML
Jenkins Credential Store	Harness Secret Manager or External Store

- ☐ **Security Rule:** Never commit secrets into Git. The pipeline references a Secret Identifier — the actual value is resolved by Harness at runtime.

Secrets Management Model

Harness supports multiple secret storage backends, making it compatible with enterprise security postures — whether you use a cloud-native store or an on-premises vault.

Supported Secret Stores



Harness Secret Manager

Built-in, encrypted secret storage within the Harness Platform



HashiCorp Vault

Enterprise on-premises or cloud Vault integration



Cloud Secret Stores

AWS Secrets Manager, Azure Key Vault, GCP Secret Manager

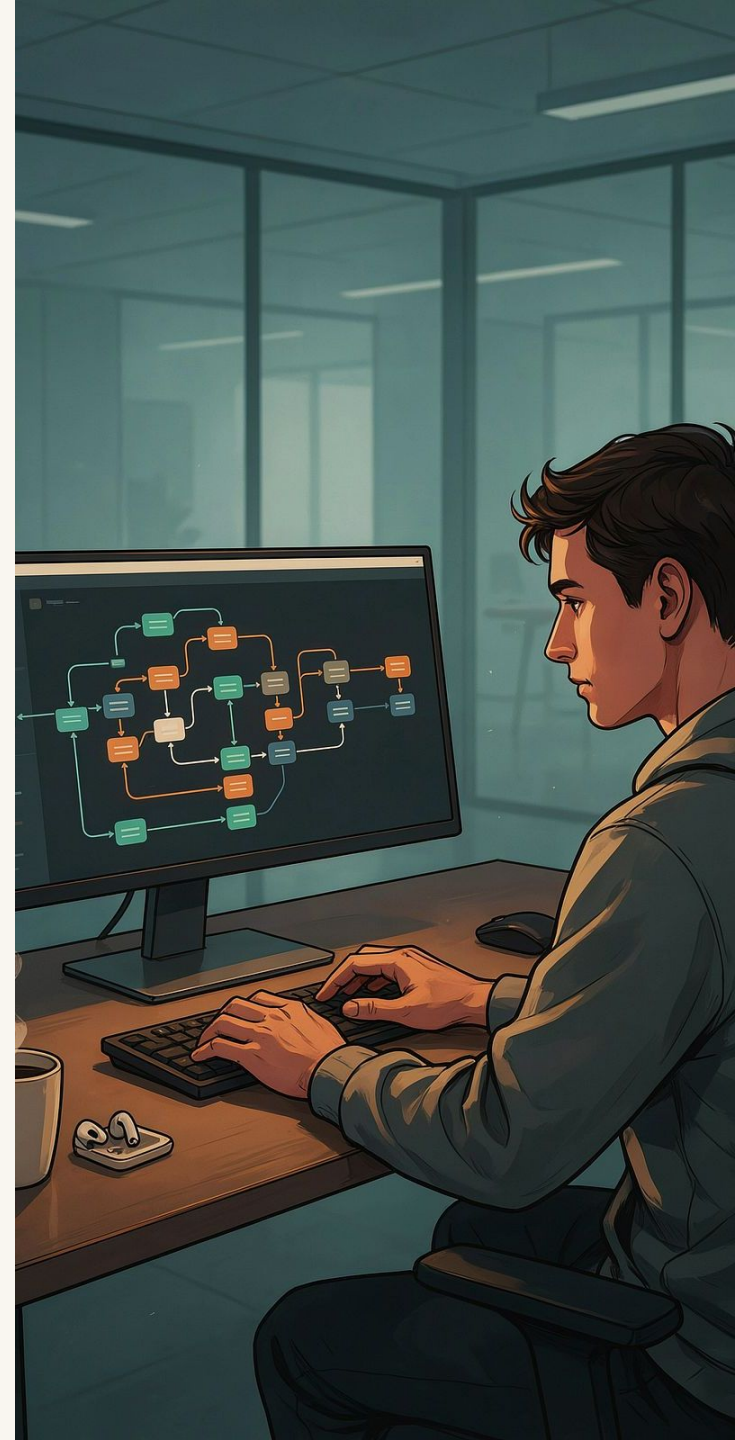
How It Works in a Pipeline

- Pipeline YAML references a **Secret Identifier** — not the value
- At runtime, Harness resolves the identifier against the configured Secret Store
- The resolved value is passed securely to the Delegate for execution
- Secret values are **never logged** in execution output

Harness Pipelines

What should happen, in what order, and under what conditions?

A Harness Pipeline is the orchestration unit — equivalent to the Jenkinsfile, but defined as structured YAML with a visual representation in the Harness UI.



Pipeline Structure: Stages & Steps

Harness Pipeline Anatomy

1

Pipeline

Top-level container — one pipeline per application or workflow

2

Stage

A logical phase — CI Build, Deploy, Test. Each Stage runs independently

3

Step

An individual operation within a Stage — Clone, Maven Build, Docker Push

Example pipeline for a Spring Boot application:

CI Stage: Clone Code → Maven Build → Unit Test → Docker Build

Deploy Stage: Kubernetes Apply → Verify

Jenkins Equivalent Structure

```
pipeline {
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Docker') {
      steps {
        sh 'docker build .'
      }
    }
  }
}
```

The hierarchy is identical: **Pipeline** → **Stage** → **Step**. The terminology maps directly.

Jenkinsfile vs. Harness Pipeline YAML

Both Jenkins and Harness support **Pipeline as Code** — the pipeline definition lives in version control alongside the application. The syntax differs, but the philosophy is the same.

Jenkins — Jenkinsfile (Groovy DSL)

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Docker Build') {
      steps {
        sh 'docker build -t app:latest .'
      }
    }
    stage('Trivy Scan') {
      steps {
        sh 'trivy image app:latest'
      }
    }
  }
}
```

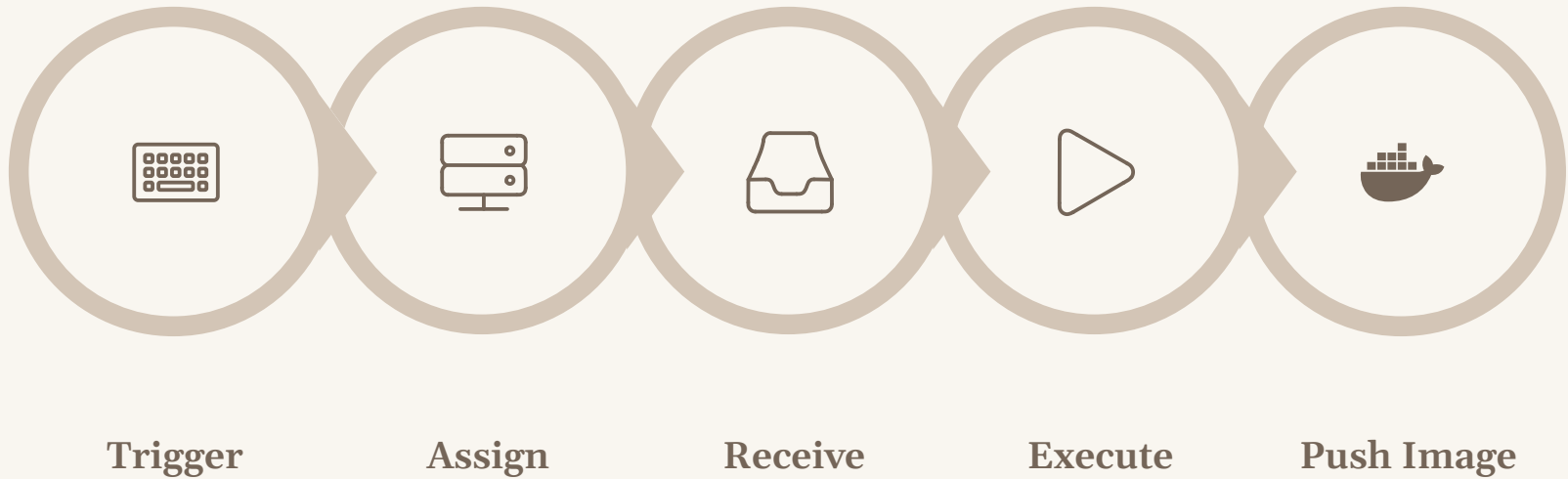
Harness — Pipeline YAML

```
pipeline:
  name: student-management-ci
  stages:
    - stage:
        name: Build
        steps:
          - step:
              name: Maven Build
              type: Run
              spec:
                command: mvn clean package
    - step:
        name: Unit Test
        type: Run
        spec:
          command: mvn test
    - stage:
        name: Package
        steps:
          - step:
              name: Docker Build
    - step:
        name: Trivy Scan
```

📌 **Key Idea:** Both files can be stored in GitHub alongside your source code — true Pipeline as Code in both ecosystems.

Pipeline Execution Flow

When a pipeline runs, it's important to understand **where each command actually executes**. This distinction is critical when working with private EKS Anywhere infrastructure.



The Critical Question for EKS Anywhere

Where does the command actually run?

Answer: The Delegate — running inside your private EKS Anywhere cluster — executes all pipeline steps. Maven runs on the Delegate. Docker builds on the Delegate. The Kubernetes API is called from the Delegate. The Harness Platform only orchestrates what happens and in what order.

Why This Matters

- Your source code never leaves your network
- Build artifacts stay inside the private environment
- Kubernetes API calls are internal — not traversing the internet

Delegate Selection in Multi-Environment Organizations

In real organizations, you'll have multiple Delegates — one per environment. **Delegate Selectors** let pipelines target the correct Delegate, just like Jenkins Agent Labels target specific build nodes.

Multi-Delegate Architecture

Delegate — Dev

Labels: dev, eks-anywhere-dev

Delegate — Test

Labels: test, eks-anywhere-test

Delegate — Production

Labels: production, eks-anywhere, onprem

Jenkins vs. Harness Targeting

Jenkins	Harness
<code>agent { label 'eks-anywhere' }</code>	Delegate Selector: <code>eks-anywhere</code>
Agent Label in Jenkinsfile	Delegate Selector in Pipeline YAML

In the pipeline YAML, specify:

```
delegateSelectors:  
- eks-anywhere  
- production
```

Harness will route execution only to Delegates matching all specified selectors.



Harness Templates

Standardize Once. Reuse Everywhere.

Imagine maintaining 50 pipelines that all contain the same Git checkout, Maven build, unit test, Trivy scan, and Docker build steps. Templates solve this at scale.

Jenkins Shared Libraries vs. Harness Templates

This is one of the most important comparisons for engineers who already use Shared Libraries in Jenkins. The concept is identical — the implementation is different.

Jenkins Shared Libraries

```
// Jenkinsfile
@Library('company-library') _
buildJavaApplication(
  mavenVersion: '3.9',
  trivyEnabled: true,
  dockerRegistry: 'registry.company.com'
)
```

The buildJavaApplication function lives in a centralized Git repository. Changes to the library propagate to all pipelines using it.

Harness Templates

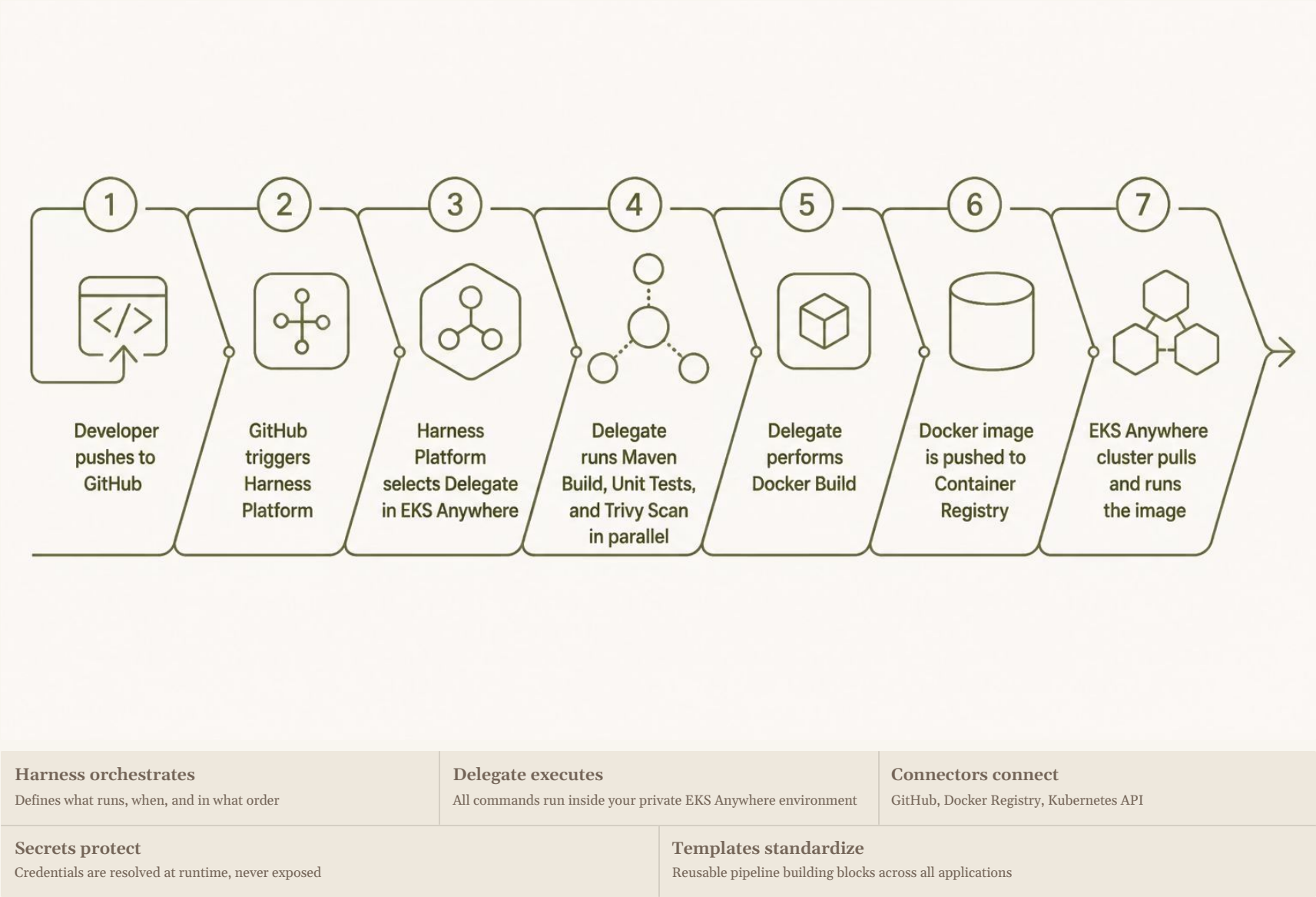
```
template:
  name: Java CI Template
  versionLabel: v1.2
  type: Pipeline
  steps:
    - step:
      name: Maven Build
      type: Run
    - step:
      name: Trivy Scan
      type: Security
    - step:
      name: Docker Build
      type: BuildAndPush
```

The Template is versioned and stored in the Harness Platform. Pipelines reference it by name and version.

Concept	Jenkins	Harness
Reusable Logic Unit	Shared Library	Template
Usage in Pipeline	@Library + function call	Template reference by name/version
Centralized Updates	Git repository commit	Template version update in Platform
Scope	Pipeline steps & stages	Steps, Stages, or full Pipelines

Complete Harness CI Flow for EKS Anywhere

This is the practical end-to-end architecture for your EKS Anywhere batch. Every component you have learned in this module appears in this flow.



Jenkins → Harness Translation Cheat Sheet

Use this as your quick-reference guide whenever you encounter a Jenkins concept and need to find its Harness equivalent.

If you know Jenkins...	Think Harness...
Jenkins Controller	Harness Platform
Jenkins Agent	Delegate
Jenkinsfile	Harness Pipeline YAML
Pipeline Stage	Stage
Pipeline Step	Step
Credentials	Secrets
SCM Configuration	Connector
Plugin Integration	Connector / Native Integration
Agent Label	Delegate Selector
Shared Library	Template
Build Parameters	Pipeline Variables
Console Output	Execution Details
Job	Pipeline
Multibranch Pipeline	Git-based Pipeline Strategy

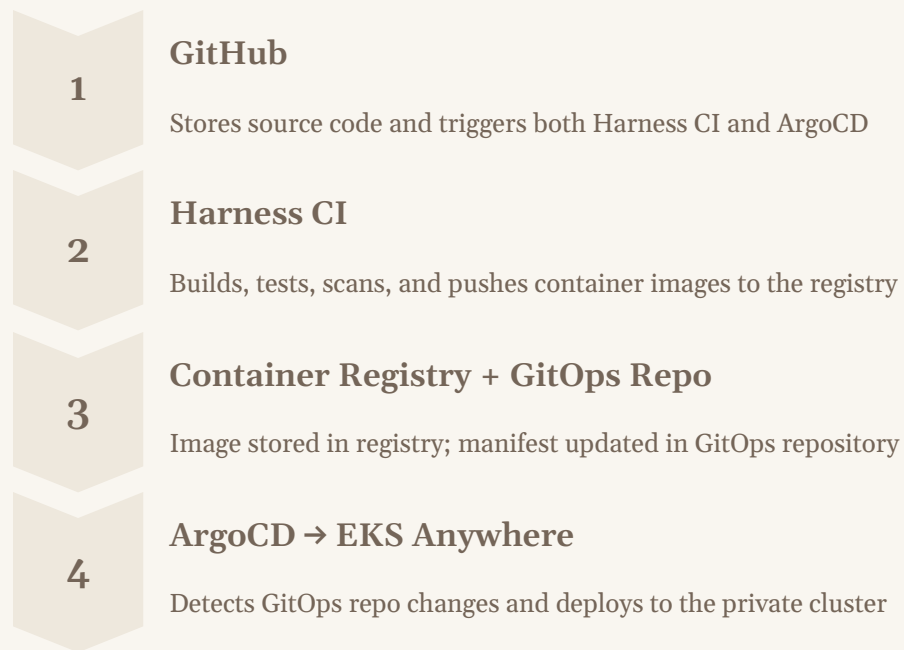
Where Harness Fits in Your EKS Anywhere Architecture

Your batch already knows this stack. Now it's time to see exactly where Harness CI plugs in — and how it complements ArgoCD rather than replacing it.

What You Already Know

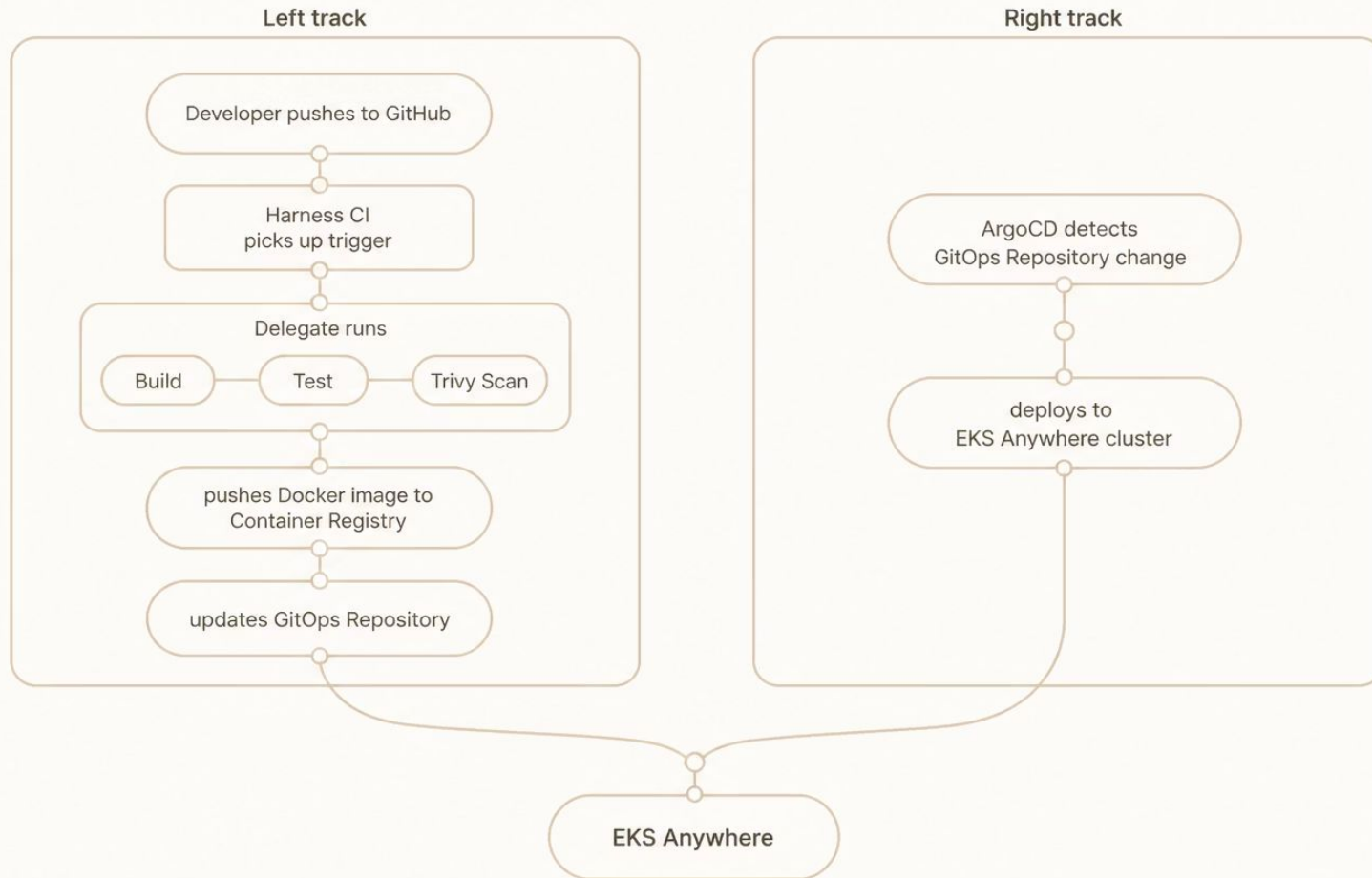
- **Git** — Source code version control
- **Docker** — Container image build and packaging
- **Kubernetes** — Container orchestration
- **EKS Anywhere** — On-premises Kubernetes distribution
- **ArgoCD** — GitOps-based continuous delivery

Adding Harness CI to the Stack



Harness CI + ArgoCD: The Complete Architecture

This is the recommended architecture for your training. It allows students to keep everything they learned about ArgoCD and understand that CI and CD do not have to be performed by the same tool.



Harness CI = Build & Package

- Clone source from GitHub
- Maven build and unit tests
- Trivy container image scan
- Docker build and push to Container Registry
- Update image tag in GitOps repository

ArgoCD = GitOps Delivery

- Monitors the GitOps repository for changes
- Detects updated image tag
- Syncs Kubernetes manifests to EKS Anywhere
- Provides deployment visibility and rollback

☐ **Architectural Insight:** Harness = CI. ArgoCD = GitOps CD. Students already learned ArgoCD — Harness CI extends that knowledge rather than replacing it.

Module 8 Complete

Harness CI — From Jenkins to Cloud-Native Delivery

Harness Platform

Cloud-hosted orchestration — replaces the Jenkins Controller

Delegate

Runs inside EKS Anywhere — executes all pipeline operations inside your private network

Connectors + Secrets

Explicit, reusable integrations with clean secret management

Pipelines + Templates

Pipeline as Code with reusable, versioned Templates across all applications

Harness + ArgoCD

CI and GitOps CD as complementary tools in your EKS Anywhere stack